

Roll number: 2403A51457

## AI-Assisted Coding Assignment(11.2)

### TASK-1:

```
1 class Stack:
2     """
3     A simple Stack implementation using a Python list.
4     Supports push, pop, peek, and is_empty operations.
5     """
6
7     def __init__(self):
8         """Initialize an empty stack."""
9         self.items = []
10
11     def push(self, item):
12         """
13         Add an item to the top of the stack.
14
15         Args:
16             item: The item to be added.
17         """
18         self.items.append(item)
19
20     def pop(self):
21         """
22         Remove and return the top item from the stack.
23
24         Returns:
25             The item at the top of the stack.
26         """
```

Generate a python program to Stack class with push, pop, peek, and is\_empty methods. Sample input is class Stack: pass and expected output should be functional stack implementation with all required methods and docstrings

```
class Stack:
    """
    A simple Stack implementation using a Python list.
    Supports push, pop, peek, and is_empty operations.
    """

    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """
        Add an item to the top of the stack.

        Args:
            item: The item to be added.
        """
        self.items.append(item)
```

PS C:\Users\DELL\AI assisted coding> & "C:\Program Files\Python313\python.exe" "c:/Users/DELL/AI assisted coding/TASK-1.py"  
Is stack empty? True  
Top item: 30  
Pop item: 30  
Top item after pop: 20  
Is stack empty? False  
PS C:\Users\DELL\AI assisted coding>

### TASK-2:

```
1 class Queue:
2
3     def peek(self):
4         """
5         Return the item at the front of the queue without removing it.
6         """
7         if not self.items:
8             raise IndexError("peek from empty queue")
9         return self.items[0]
10
11     def size(self):
12         """
13         Return the number of items in the queue.
14         """
15         return len(self.items)
16
17 if __name__ == "__main__":
18     q = Queue()
19     q.enqueue(10)
20     q.enqueue(20)
21     q.enqueue(30)
22     print("Queue size:", q.size()) # Output: 3
23     print("Front item:", q.peek()) # Output: 10
24     print("Dequeued:", q.dequeue()) # Output: 10
25     print("Queue size:", q.size()) # Output: 2
```

Implement a Queue using Python lists. Sample Input Code: class Queue: pass Expected Output: • FIFO-based queue class with enqueue, dequeue, peek, and size methods

```
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        """Add item to the end of the queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the item from the front
        if not self.items:
            raise IndexError("dequeue from empty queue")
        return self.items.pop(0)

    def peek(self):
        """Return the item at the front of the queue
        if not self.items:
            raise IndexError("peek from empty queue")
        return self.items[0]
```

PS C:\Users\DELL\AI assisted coding> & "C:\Program Files\python313\python.exe" "c:/Users/DELL/AI assisted coding/TASK-2"  
Queue size: 3  
Front item: 10  
Dequeued: 10  
Queue size: 2  
PS C:\Users\DELL\AI assisted coding>

## TASK-3:

The screenshot shows the Visual Studio Code interface with the 'TASK-3' file open. The code defines a `Node` class and a `LinkedList` class. The `Node` class has an `__init__` method that takes `data` and sets `self.data = data` and `self.next = None`. The `LinkedList` class has an `__init__` method that sets `self.head = None` and an `insert` method that adds a new node at the end of the list. The terminal shows the execution of the program, which prints the linked list contents: 10 20 30 40 50 60 70 80 900.

```
class Node:
    """
    Node class represents an element in the singly linked list.
    Each node contains data and a reference to the next node.
    """
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    """
    LinkedList class provides methods to insert elements and display the list
    """
    def __init__(self):
        self.head = None

    def insert(self, data):
        """
        Inserts a new node with the given data at the end of the list.
        """
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        current = self.head
```

Terminal output:

```
PS C:\Users\DELL\AI assisted coding> & "C:\Program Files\Python313\python.exe" "c:/Users/DELL/AI as
sisted coding/TASK-3"
LinkedList contents:
10 20 30 40 50 60 70 80 900
PS C:\Users\DELL\AI assisted coding>
```

## TASK-4:

The screenshot shows the Visual Studio Code interface with the 'TASK-4' file open. The code defines a `Node` class and a `BST` class. The `Node` class has an `__init__` method that takes `key` and sets `self.key = key`, `self.left = None`, and `self.right = None`. The `BST` class has an `__init__` method that sets `self.root = None` and an `insert` method that inserts a new node into the tree. The terminal shows the execution of the program, which prints the in-order traversal of the BST: 10 20 30 40 50 60 70 80 90.

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

    def insert(self, key):
        self.root = self._insert_recursive(self.root, key)

    def _insert_recursive(self, node, key):
        if node is None:
            return Node(key)
        if key < node.key:
            node.left = self._insert_recursive(node.left, key)
        elif key > node.key:
            node.right = self._insert_recursive(node.right, key)
        return node

    def in_order_traversal(self):
        self._in_order_recursive(self.root)
        print() # For newline after traversal
```

Terminal output:

```
PS C:\Users\DELL\AI assisted coding> & "C:\Program Files\Python313\python.exe" "c:/Users/DELL/AI as
sisted coding/TASK-4"
In-order Traversal:
10 20 30 40 50 60 70 80 90
PS C:\Users\DELL\AI assisted coding>
```

## TASK-5:

The screenshot shows the Visual Studio Code interface with the Explorer, AI ASSISTED CODING, and Terminal panels. The AI ASSISTED CODING panel displays the code for Task 5, which is a HashTable implementation. The code includes methods for initialization, hashing, insertion, and searching. The Terminal panel shows the execution of the code, which successfully inserts and searches for values in the hash table.

```
class HashTable:
    def __init__(self, size=10):
        # Initialize the hash table with empty buckets (lists)
        self.size = size
        self.table = [[] for _ in range(self.size)]

    def _hash(self, key):
        # Simple hash function using built-in hash and modulo
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        index = self._hash(key)
        bucket = self.table[index]
        # Check if key exists and update
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                return
        # If key does not exist, append to the bucket (chaining)
        bucket.append((key, value))

    def search(self, key):
        """Search for a value by key in the hash table."""
        index = self._hash(key)
```

Terminal output:

```
PS C:\Users\DELL\AI assisted coding> & "C:\Program Files\Python313\python.exe" "c:/Users/DELL/AI as
sisted coding/TASK-5"
Search 'banana': 2
After deleting 'banana': None
Hash Table: [[], [], [], [], [], [], [('orange', 3)], [], [], [('apple', 1)]]
PS C:\Users\DELL\AI assisted coding>
```

## TASK-6:

The screenshot shows the Visual Studio Code interface with the Explorer, AI ASSISTED CODING, and Terminal panels. The AI ASSISTED CODING panel displays the code for Task 6, which is a Graph implementation using an adjacency list. The code includes methods for adding vertices, adding edges, and displaying the graph. The Terminal panel shows the execution of the code, which successfully adds vertices and edges to the graph.

```
class Graph:
    def __init__(self):
        self.adj_list = {}

    def add_vertex(self, vertex):
        if vertex not in self.adj_list:
            self.adj_list[vertex] = []

    def add_edge(self, vertex1, vertex2):
        if vertex1 not in self.adj_list:
            self.add_vertex(vertex1)
        if vertex2 not in self.adj_list:
            self.add_vertex(vertex2)
        self.adj_list[vertex1].append(vertex2)
        self.adj_list[vertex2].append(vertex1) # For undirected graph

    def display(self):
        for vertex in self.adj_list:
            print(f"{vertex}: {self.adj_list[vertex]}")

# Example usage:
if __name__ == "__main__":
    g = Graph()
    g.add_vertex("A")
    g.add_vertex("B")
```

Terminal output:

```
PS C:\Users\DELL\AI assisted coding> & "C:\Program Files\Python313\python.exe" "c:/Users/DELL/AI as
sisted coding/TASK-6"
A: ['B', 'C']
B: ['A', 'D']
C: ['A']
D: ['B']
PS C:\Users\DELL\AI assisted coding>
```

## TASK-7:

The screenshot shows the Visual Studio Code interface with the Explorer, Search, and Run and Debug toolbars. The Explorer sidebar on the left shows a project named 'AI ASSISTED CODING' with files TASK-1.py through TASK-7. The main editor window displays the code for TASK-7, which implements a Priority Queue using Python's heapq module. The code includes a class PriorityQueue with methods \_\_init\_\_, enqueue, dequeue, and display. The enqueue method uses heapq.heappush to add items with negative priorities. The dequeue method uses heapq.heappop to remove the item with the highest priority (lowest negative value). The display method sorts the heap and returns the items. The terminal window at the bottom shows the execution of the code, which outputs the tasks in priority order: Task B, Task A, and Task C.

```
1 import heapq
2
3 class PriorityQueue:
4     def __init__(self):
5         self._heap = []
6
7     def enqueue(self, item, priority):
8         # heapq is a min-heap, so use negative priority for max-heap behavior
9         heapq.heappush(self._heap, (-priority, item))
10
11     def dequeue(self):
12         if not self._heap:
13             return None
14         return heapq.heappop(self._heap)[1]
15
16     def display(self):
17         # Show items sorted by priority (highest first)
18         return [item for _, item in sorted(self._heap, reverse=True)]
19
20 # Example usage:
21 if __name__ == "__main__":
22     pq = PriorityQueue()
23     pq.enqueue("Task A", 2)
24     pq.enqueue("Task B", 5)
25     pq.enqueue("Task C", 1)
```

PS C:\Users\DELL\AI assisted coding> & "C:\Program Files\Python313\python.exe" "c:/Users/DELL/ai assisted coding/TASK-7"
Queue: ['Task C', 'Task A', 'Task B']
Dequeue: Task B
Queue after dequeue: ['Task C', 'Task A']
PS C:\Users\DELL\AI assisted coding>

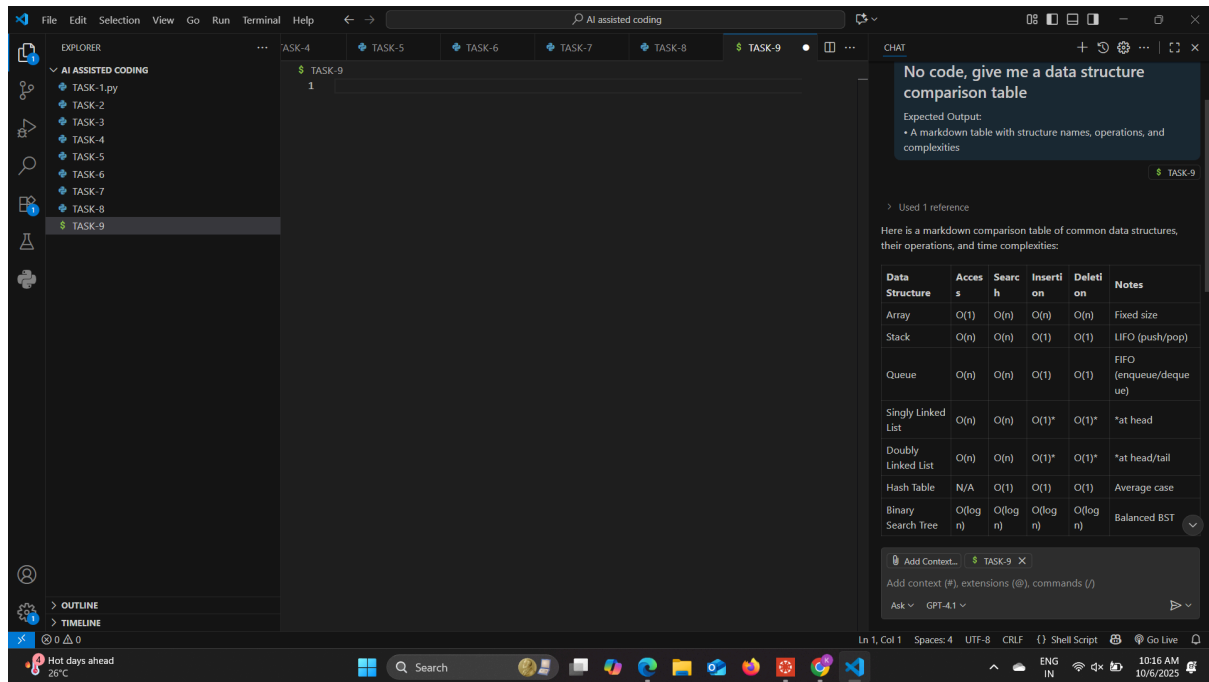
## TASK-8:

The screenshot shows the Visual Studio Code interface with the Explorer, Search, and Run and Debug toolbars. The Explorer sidebar on the left shows a project named 'AI ASSISTED CODING' with files TASK-1.py through TASK-8. The main editor window displays the code for TASK-8, which implements a Double-Ended Queue using Python's collections.deque. The code includes a class DequeDS with methods \_\_init\_\_, insert\_front, insert\_rear, and remove\_front. The insert\_front method uses deque.appendleft to add items to the front. The insert\_rear method uses deque.append to add items to the rear. The remove\_front method uses deque.popleft to remove items from the front. The terminal window at the bottom shows the execution of the code, which outputs the tasks in priority order: Task B, Task A, and Task C.

```
1 from collections import deque
2
3 class DequeDS:
4     """
5     Double-ended queue implementation using collections.deque.
6     Supports insertion and removal from both ends.
7     """
8
9     def __init__(self):
10         """Initialize an empty deque."""
11         self.deque = deque()
12
13     def insert_front(self, item):
14         """Insert an item at the front of the deque."""
15         self.deque.appendleft(item)
16         print(f"Inserted {item} at front: {list(self.deque)}")
17
18     def insert_rear(self, item):
19         """Insert an item at the rear of the deque."""
20         self.deque.append(item)
21         print(f"Inserted {item} at rear: {list(self.deque)}")
22
23     def remove_front(self):
24         """Remove and return the item from the front of the deque."""
25         if self.deque:
```

PS C:\Users\DELL\AI assisted coding> & "C:\Program Files\Python313\python.exe" "c:/Users/DELL/ai assisted coding/TASK-8"
Inserted 10 at front: [10]
Inserted 20 at rear: [10, 20]
Inserted 5 at front: [5, 10, 20]
Removed 20 from rear: [5, 10]
Removed 5 from front: [10]
PS C:\Users\DELL\AI assisted coding>

## TASK-9:



File Edit Selection View Go Run Terminal Help AI assisted coding

EXPLORER AI ASSISTED CODING TASK-1.py TASK-2 TASK-3 TASK-4 TASK-5 TASK-6 TASK-7 TASK-8 TASK-9

CHAT

No code, give me a data structure comparison table

Expected Output:  
• A markdown table with structure names, operations, and complexities

Used 1 reference

Here is a markdown comparison table of common data structures, their operations, and time complexities:

| Data Structure     | Access   | Search   | Insertion | Deletion | Notes                  |
|--------------------|----------|----------|-----------|----------|------------------------|
| Array              | O(1)     | O(n)     | O(n)      | O(n)     | Fixed size             |
| Stack              | O(n)     | O(n)     | O(1)      | O(1)     | LIFO (push/pop)        |
| Queue              | O(n)     | O(n)     | O(1)      | O(1)     | FIFO (enqueue/dequeue) |
| Singly Linked List | O(n)     | O(n)     | O(1)*     | O(1)*    | *at head               |
| Doubly Linked List | O(n)     | O(n)     | O(1)*     | O(1)*    | *at head/tail          |
| Hash Table         | N/A      | O(1)     | O(1)      | O(1)     | Average case           |
| Binary Search Tree | O(log n) | O(log n) | O(log n)  | O(log n) | Balanced BST           |

Add Context... TASK-9 X

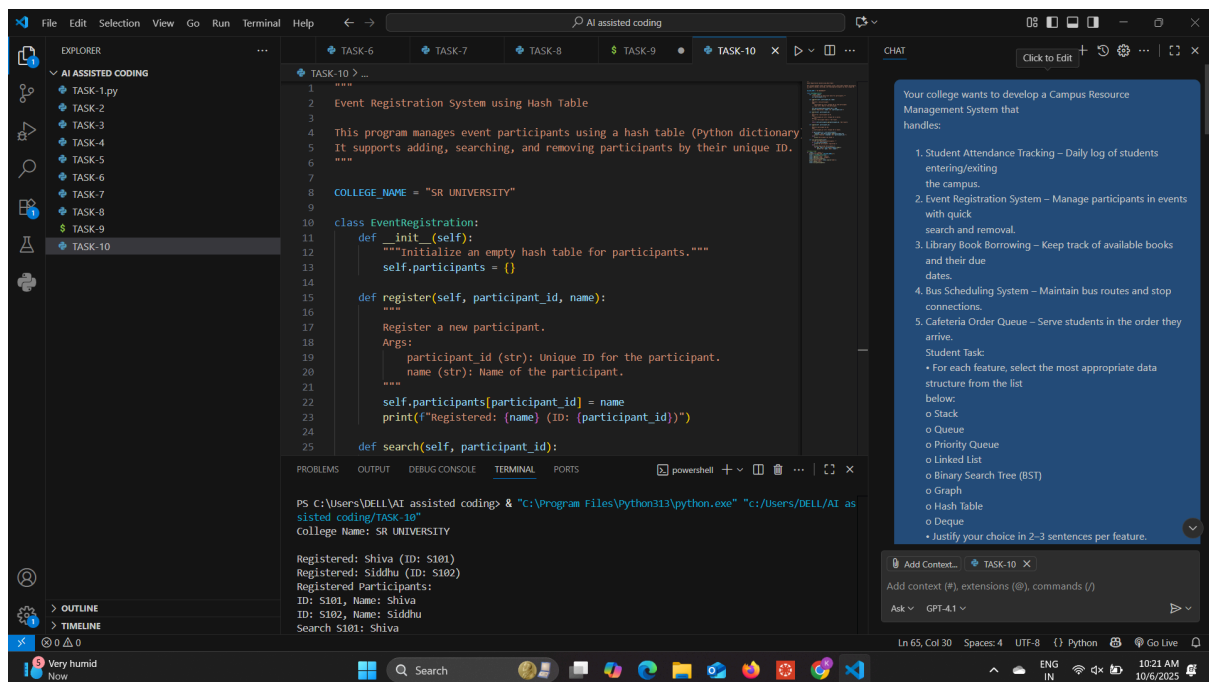
Add context (#), extensions (@), commands (/)

Ask GPT-4.1

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Shell Script Go Live

Hot days ahead 26°C

## TASK-10:



File Edit Selection View Go Run Terminal Help AI assisted coding

EXPLORER AI ASSISTED CODING TASK-1.py TASK-2 TASK-3 TASK-4 TASK-5 TASK-6 TASK-7 TASK-8 TASK-9 TASK-10

CHAT

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:  
• For each feature, select the most appropriate data structure from the list below:  
o Stack  
o Queue  
o Priority Queue  
o Linked List  
o Binary Search Tree (BST)  
o Graph  
o Hash Table  
o Deque  
• Justify your choice in 2-3 sentences per feature.

Add Context... TASK-10 X

Add context (#), extensions (@), commands (/)

Ask GPT-4.1

Ln 65, Col 30 Spaces: 4 UTF-8 Python Go Live

Very humid Now

PS C:\Users\DELL\AI assisted coding> & "C:\Program Files\Python313\python.exe" "c:/Users/DELL/AI assisted coding/TASK-10"

College Name: SR UNIVERSITY

Registered: Shiva (ID: S101)  
Registered: Siddhu (ID: S102)  
Registered Participants:  
ID: S101, Name: Shiva  
ID: S102, Name: Siddhu  
Search S101: Shiva